

Module 04: Cognition — Implementing the T-Maze

Course: Active Inference 101 | **Unit:** Implementation | **Audience:** First-semester undergraduates

Learning Objectives

1. Implement the classic **T-Maze** environment — the standard benchmark for Active Inference.
2. Set up the **full POMDP** with multiple observation modalities and state factors.
3. Demonstrate how the agent combines **information-seeking** and **goal-seeking** behavior.

Introduction

The T-Maze is the "Hello World" of Active Inference. An agent starts in a central corridor and must choose to go left or right. One arm has a reward, the other doesn't. A cue at the start tells the agent (imperfectly) where the reward is.

Key Concepts

1. T-Maze Setup

```

import numpy as np

def build_tmaze(reward_prob=0.8, cue_reliability=0.9):
    """
    Build the T-Maze POMDP.

    States: 4 locations × 2 reward conditions = 8 states
    - Locations: center, cue, left_arm, right_arm
    - Reward: left or right

    Observations: 3 modalities
    - Location: where am I? (4 outcomes)
    - Cue: what does the cue say? (3: left, right, no_cue)
    - Reward: did I get rewarded? (3: reward, loss, neutral)
    """
    num_locations = 4 # center, cue, left, right
    num_reward_conds = 2 # reward_left, reward_right
    num_states = num_locations * num_reward_conds # 8

    # --- A matrices (one per observation modality) ---

    # A[0]: Location observation (4 × 8) – always accurate
    A_location = np.zeros((4, num_states))
    for loc in range(4):
        for rc in range(2):
            A_location[loc, loc * 2 + rc] = 1.0

    # A[1]: Cue observation (3 × 8)
    A_cue = np.zeros((3, num_states))
    for loc in range(4):
        for rc in range(2):

```

```

s = loc * 2 + rc
if loc == 1: # at cue location
    if rc == 0: # reward is left
        A_cue[0, s] = cue_reliability # cue says "left"
        A_cue[1, s] = 1 - cue_reliability
    else:
        A_cue[1, s] = cue_reliability # cue says "right"
        A_cue[0, s] = 1 - cue_reliability
else:
    A_cue[2, s] = 1.0 # no cue visible

# A[2]: Reward observation (3 × 8)
A_reward = np.zeros((3, num_states))
for loc in range(4):
    for rc in range(2):
        s = loc * 2 + rc
        if loc == 2: # left arm
            if rc == 0: # reward is left
                A_reward[0, s] = reward_prob # reward
                A_reward[1, s] = 1 - reward_prob
            else:
                A_reward[1, s] = reward_prob # loss
                A_reward[0, s] = 1 - reward_prob
        elif loc == 3: # right arm
            if rc == 1: # reward is right
                A_reward[0, s] = reward_prob
                A_reward[1, s] = 1 - reward_prob
            else:
                A_reward[1, s] = reward_prob
                A_reward[0, s] = 1 - reward_prob
        else:
            A_reward[2, s] = 1.0 # neutral

```

```

A = [A_location, A_cue, A_reward]

# --- B matrix (8 × 8 × 4 actions) ---
# Actions: stay, go_cue, go_left, go_right
B = np.zeros((num_states, num_states, 4))

# Transition logic: actions move between locations
# Reward condition stays the same
action_targets = {0: None, 1: 1, 2: 2, 3: 3} # stay, cue, left, right

for a, target in action_targets.items():
    for rc in range(2):
        for loc in range(4):
            s_from = loc * 2 + rc
            if target is None:
                s_to = s_from # stay
            else:
                s_to = target * 2 + rc
            B[s_to, s_from, a] = 1.0

# --- C vectors (preferences per modality) ---
C_location = np.zeros(4)
C_cue = np.zeros(3)
C_reward = np.array([3.0, -3.0, 0.0]) # prefer reward, avoid loss
C = [C_location, C_cue, C_reward]

# --- D vector (initial state prior) ---
D = np.zeros(num_states)
D[0] = 0.5 # start at center, reward_left
D[1] = 0.5 # start at center, reward_right

```

```
return A, B, C, D
```

2. Running the T-Maze Agent

```
A, B, C, D = build_tmaze()  
# With the full POMDP, the agent should:  
# 1. First visit the cue (information-seeking)  
# 2. Then go to the rewarded arm (goal-seeking)
```

3. Expected Behavior

The classic T-Maze result demonstrates Active Inference's exploration-exploitation:

1. **Step 1:** Agent goes to the cue → epistemic action (gathering information)
2. **Step 2:** Agent goes to the rewarded arm → pragmatic action (achieving goals)

This emerges naturally from EFE without any explicit exploration strategy.

Summary

The T-Maze is the canonical benchmark for Active Inference, demonstrating how EFE naturally balances information-seeking and goal-seeking behavior. Building it requires multi-modal observations and careful POMDP construction.

Further Reading

- Friston, K. J. et al. (2015). Active inference and epistemic value. *Cognitive Neuroscience*, 6(4), 187-214.
- Smith, R. et al. (2022). A step-by-step tutorial on active inference. *Journal of Mathematical Psychology*, 107, 102632.